



Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Advanced Digital Systems Design
ENCS3310

Design and Verification of a Simplified Memory Controller

Prepared by:

Name: Batol Abu Samhadana

ID: 1230738

Section: 1

Instructor: Dr. Ayman Hroub

Date: 18/9/2025

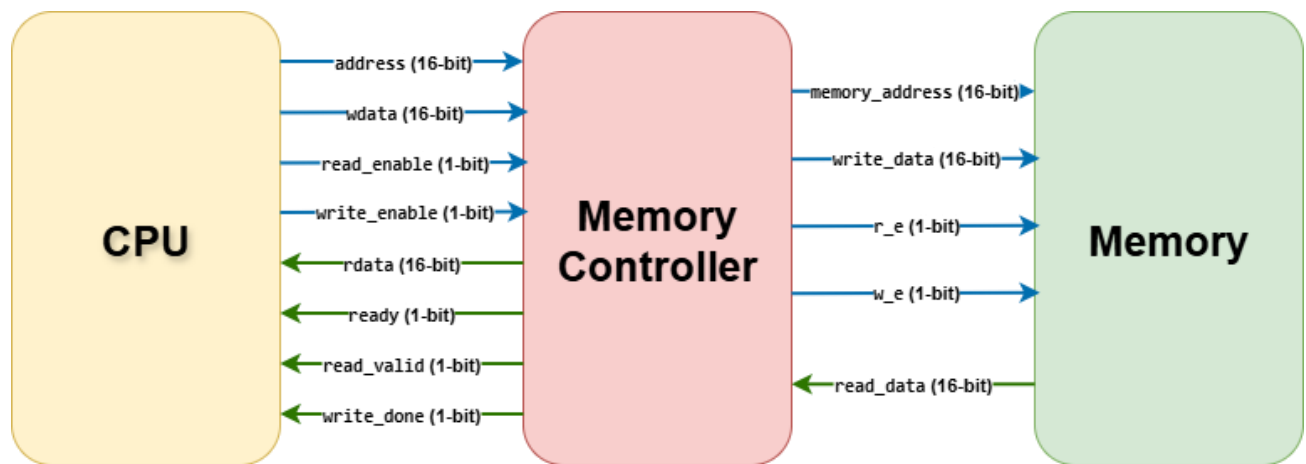
Introduction

This project presents the design and verification of a simplified synchronous memory controller that manages data transfer between a CPU model and a memory module. The controller supports read and write operations with write requests having higher priority. It uses a simple FSM with three states—idle, read, and write—to control the flow of operations. A basic CPU model is implemented to generate requests and receive responses, while a memory model stores the data. Handshake signals (ready, read_valid, and write_done) ensure proper synchronization between the CPU and controller. The design is verified using comprehensive Verilog testbenches for both the memory controller alone and the complete system. Simulation results confirm correct functionality across different scenarios.

Table of Contents

Introduction	2
System Block Diagram.....	4
Memory Controller RTL Code.....	5
Memory Controller Testbench Code.....	8
Memory Controller Simulation Screenshots	10
Complete System RTL Code.....	11
Complete System Testbench Code.....	14
Complete System Simulation Screenshots.....	16
Conclusion	17

System Block Diagram



Memory Controller RTL Code

```
94 module memory_controller (  
95     clk,  
96     rst_n,  
97     address,  
98     wdata,  
99     rdata,  
100    read_enable,  
101    write_enable,  
102    ready,  
103    read_valid,  
104    write_done,  
105    memory_address,  
106    write_data,  
107    read_data,  
108    r_e,  
109    w_e  
110 );  
111 input clk, rst_n;  
112  
113 // CPU interface  
114 input [15:0] address, wdata;  
115 input read_enable, write_enable;  
116 output reg [15:0] rdata;  
117 output reg ready, read_valid, write_done;  
118  
119 // Memory interface  
120 input [15:0] read_data;  
121 output reg [15:0] memory_address, write_data;  
122 output reg r_e, w_e;  
123  
124 // state machine encoding  
125 reg [1:0] state, next_state;  
126 parameter S_IDLE = 2'b00, S_READ = 2'b01, S_WRITE = 2'b10;  
127  
128 reg pending_read; // flag in case user did read and write together  
129 reg [15:0] pending_read_address;  
130 reg [1:0] read_cycle_cnt;  
131  
132 always @(posedge clk or negedge rst_n) begin  
133     if (!rst_n) begin // reset zero everything  
134         state <= S_IDLE;  
135         ready <= 1'b1;  
136         read_valid <= 1'b0;
```

```

137     write_done <= 1'b0;
138     r_e <= 1'b0;
139     w_e <= 1'b0;
140     pending_read <= 1'b0;
141     read_cycle_cnt <= 2'b00;
142     memory_address <= 16'b0;
143     write_data <= 16'b0;
144     rdata <= 16'b0;
145 end else begin
146     state <= next_state;
147     read_valid <= 1'b0;
148     write_done <= 1'b0;
149
150     case (state)
151     S_IDLE: begin
152         r_e <= 1'b0;
153         w_e <= 1'b0;
154         ready <= 1'b1;
155         read_cycle_cnt <= 2'b00;
156         if (write_enable) begin
157             memory_address <= address;
158             write_data <= wdata;
159         end else if (read_enable) begin
160             memory_address <= address;
161         end
162     end
163     S_WRITE: begin
164         w_e <= 1'b1;
165         r_e <= 1'b0;
166         ready <= 1'b0;
167         write_done <= 1'b1;
168     end
169     S_READ: begin
170         r_e <= 1'b1;
171         w_e <= 1'b0;
172         ready <= 1'b0;
173         read_cycle_cnt <= read_cycle_cnt + 1'b1;
174         if (read_cycle_cnt == 2'b10) begin
175             rdata <= read_data;
176             read_valid <= 1'b1;
177         end
178     end
179 endcase

```

```

180     end
181 end
182
183 always @(*) begin
184     next_state = state;
185     case (state)
186     S_IDLE: begin
187         if (write_enable) next_state = S_WRITE;
188         else if (read_enable) next_state = S_READ;
189     end
190     S_WRITE: begin
191         if (pending_read) next_state = S_READ;
192         else next_state = S_IDLE;
193     end
194     S_READ: begin
195         if (read_cycle_cnt == 2'b10) next_state = S_IDLE;
196         else next_state = S_READ;
197     end
198     endcase
199 end
200
201 always @(posedge clk or negedge rst_n) begin
202     if (!rst_n) begin
203         pending_read <= 1'b0;
204         pending_read_address <= 16'b0;
205     end else begin
206         if (state == S_IDLE) begin
207             if (write_enable && read_enable) begin
208                 pending_read <= 1'b1;
209                 pending_read_address <= address;
210             end
211         end else if (state == S_WRITE) begin
212             if (pending_read && (next_state == S_READ)) begin
213                 pending_read <= 1'b0;
214                 memory_address <= pending_read_address;
215             end
216         end
217     end
218 end
219 endmodule

```

Memory Controller TestBench Code

```
1 `timescale 1ns / 1ns
2 module memory_controller_tb;
3
4     reg clk, rst_n;
5     reg [15:0] address, wdata;
6     reg read_enable, write_enable;
7     wire [15:0] rdata, memory_address, write_data;
8     wire ready, read_valid, write_done;
9     wire r_e, w_e;
10
11     initial begin
12         $dumpfile("memory_controller_tb.vcd"); // name of the waveform file
13         $dumpvars(0, memory_controller_tb); // dump all signals in this module
14         clk = 0;
15         end
16     always #5 clk = ~clk;
17
18     // Instantiate memory controller
19     memory_controller u_controller (
20         .clk(clk),
21         .rst_n(rst_n),
22         .address(address),
23         .wdata(wdata),
24         .rdata(rdata),
25         .read_enable(read_enable),
26         .write_enable(write_enable),
27         .ready(ready),
28         .read_valid(read_valid),
29         .write_done(write_done),
30         .memory_address(memory_address),
31         .write_data(write_data),
32         .read_data(16'h1234), // dummy memory input
33         .r_e(r_e),
34         .w_e(w_e)
35     );
36
37     initial begin
38         #0 rst_n = 0;
39         #20 rst_n = 1;
40
41         $monitor("Time=%0t Addr=%h WData=%h RData=%h Ready=%b ReadValid=%b WriteDone=%b", $time,
42             address, wdata, rdata, ready, read_valid, write_done);
43
44         wdata = 16'h0000;
45         address = 16'h0000;
46         {write_enable, read_enable} = 2'b00;
47
48         // Test multiple addresses
49         repeat (2) begin
50             #10 address = 16'h0010;
51             #10 wdata = 16'hAAAA;
52
53             repeat (4) begin
54                 #10 {write_enable, read_enable} = {write_enable, read_enable} + 2'b01;
55             end
56
57             #10 address = 16'h0020;
58             #10 wdata = 16'h5555;
59
60             repeat (4) begin
61                 #10 {write_enable, read_enable} = {write_enable, read_enable} + 2'b01;
62             end
63         end
64
65         #50 $finish;
66     end
67 endmodule
```


- TB output:

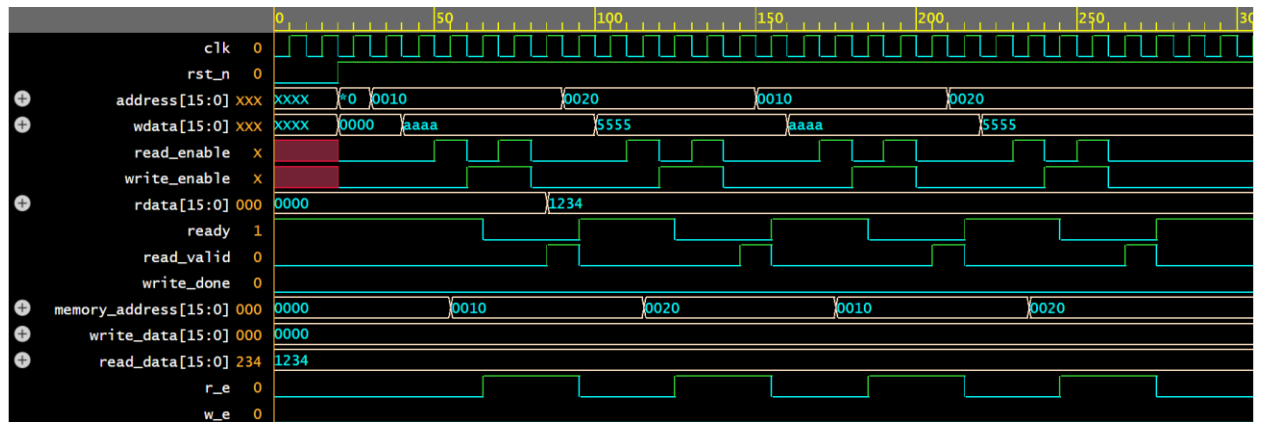
```

Time=20 Addr=0000 WData=0000 RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=30 Addr=0010 WData=0000 RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=40 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=65 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=0
Time=85 Addr=0010 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=90 Addr=0020 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=95 Addr=0020 WData=aaaa RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=100 Addr=0020 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=125 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=0 WriteDone=0
Time=145 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=150 Addr=0010 WData=5555 RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=155 Addr=0010 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=160 Addr=0010 WData=aaaa RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=185 Addr=0010 WData=aaaa RData=1234 Ready=0 ReadValid=0 WriteDone=0
Time=205 Addr=0010 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=210 Addr=0020 WData=aaaa RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=215 Addr=0020 WData=aaaa RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=220 Addr=0020 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
Time=245 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=0 WriteDone=0
Time=265 Addr=0020 WData=5555 RData=1234 Ready=0 ReadValid=1 WriteDone=0
Time=275 Addr=0020 WData=5555 RData=1234 Ready=1 ReadValid=0 WriteDone=0
$finish called from file "testbench.sv", line 65.
$finish at simulation time 310

```

Memory Controller Simulation Screenshots

- All controller signals:



Complete System RTL Code

```
221 module top_system (
222     clk,
223     rst_n
224 );
225     input clk, rst_n;
226
227     // CPU to controller signals
228     wire [15:0] cpu_address, cpu_wdata, cpu_rdata;
229     wire cpu_read_enable, cpu_write_enable;
230     wire cpu_ready, cpu_read_valid, cpu_write_done;
231
232     // controller to memory signals
233     wire [15:0] mem_address, mem_write_data, mem_read_data;
234     wire mem_r_e, mem_w_e;
235
236     // instantiate CPU
237     cpu u_cpu (
238         .clk(clk),
239         .rst_n(rst_n),
240         .address(cpu_address),
241         .wdata(cpu_wdata),
242         .rdata(cpu_rdata),
243         .read_enable(cpu_read_enable),
244         .write_enable(cpu_write_enable),
245         .ready(cpu_ready),
246         .read_valid(cpu_read_valid),
247         .write_done(cpu_write_done)
248     );
249
250     // instantiate memory
251     memory u_memory (
252         .clk(clk),
253         .rst_n(rst_n),
254         .address(mem_address),
255         .write_data(mem_write_data),
256         .read_data(mem_read_data),
257         .r_e(mem_r_e),
258         .w_e(mem_w_e)
259     );
260
261     // instantiate memory controller
262     memory_controller u_controller (
263         .clk(clk),
264
265         .rst_n(rst_n),
266         .address(cpu_address),
267         .wdata(cpu_wdata),
268         .rdata(cpu_rdata),
269         .read_enable(cpu_read_enable),
270         .write_enable(cpu_write_enable),
271         .ready(cpu_ready),
272         .read_valid(cpu_read_valid),
273         .write_done(cpu_write_done),
274         .memory_address(mem_address),
275         .write_data(mem_write_data),
276         .read_data(mem_read_data),
277         .r_e(mem_r_e),
278         .w_e(mem_w_e)
279     );
280 endmodule
```

- Simple memory code:

```

63 module memory (
64     clk,
65     rst_n,
66     address,
67     write_data,
68     read_data,
69     r_e,
70     w_e
71 );
72 input clk, rst_n;
73 input [15:0] address, write_data;
74 input r_e, w_e;
75 output reg [15:0] read_data;
76
77 reg [15:0] mem[0:65535];
78
79 always @(*) begin
80     if (r_e) read_data = mem[address];
81     else read_data = 16'b0;
82 end
83
84 always @(posedge clk or negedge rst_n) begin
85     if (!rst_n) begin
86         integer i;
87         for (i = 0; i < 65536; i = i + 1) mem[i] <= 16'b0;
88     end else begin
89         if (w_e) mem[address] <= write_data;
90     end
91 end
92 endmodule

```

- Simple CPU code:

```

1 module cpu (
2     clk,
3     rst_n,
4     address,
5     wdata,
6     rdata,
7     read_enable,
8     write_enable,
9     ready,
10    read_valid,
11    write_done
12);
13 input clk, rst_n;
14 output reg [15:0] address, wdata;
15 input [15:0] rdata;
16 output reg read_enable, write_enable;
17 input ready, read_valid, write_done;
18
19
20 reg [7:0] step;
21
22
23 always @(posedge clk or negedge rst_n) begin
24     if (!rst_n) begin
25         step <= 0;
26         address <= 0;
27         wdata <= 0;
28         read_enable <= 0;
29         write_enable <= 0;
30     end else begin
31         read_enable <= 0;
32         write_enable <= 0;
33         case (step)
34             0:
35                 if (ready) begin
36                     address <= 16'h0010;
37                     wdata <= 16'hAAAA;
38                     write_enable <= 1;
39                     step <= 1;
40                 end
41             1: if (write_done) step <= 2;
42             2:
43                 if (ready) begin
44                     address <= 16'h0010;
45                     read_enable <= 1;
46                     step <= 3;
47                 end
48             3: if (read_valid) step <= 4;
49             4:
50                 if (ready) begin
51                     address <= 16'h0020;
52                     wdata <= 16'h5555;
53                     write_enable <= 1;
54                     read_enable <= 1;
55                     step <= 5;
56                 end
57             5: if (write_done) step <= 6;
58         endcase
59     end
60 end
61 endmodule

```

Complete System TestBench Code

```
1 `timescale 1ns / 1ns
2 module top_system_tb;
3
4     reg clk, rst_n;
5     initial begin
6         $dumpfile("top_system_tb.vcd"); // name of the waveform file
7         $dumpvars(0, top_system_tb); // dump all signals in this module
8         clk = 0;
9         end
10        always #5 clk = ~clk;
11
12    top_system u_top (
13        .clk (clk),
14        .rst_n(rst_n)
15    );
16    initial begin
17        #0 rst_n = 0;
18        #20 rst_n = 1;
19
20        $monitor("Time=%0t Addr=%h WData=%h RData=%h Ready=%b ReadValid=%b WriteDone=%b", $time,
21            u_top.u_cpu.address, u_top.u_cpu.wdata, u_top.u_cpu.rdata, u_top.u_cpu.ready,
22            u_top.u_cpu.read_valid, u_top.u_cpu.write_done);
23
24        {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} = 2'b00;
25
26        // multiple addresses
27        repeat (2) begin
28            #10 u_top.u_cpu.address = 16'h0010;
29            #10 u_top.u_cpu.wdata = 16'hAAAA;
30
31            repeat (4) begin
32                #10
33                {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} =
34                    {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} + 2'b01;
35            end
36
37            #10 u_top.u_cpu.address = 16'h0020;
38            #10 u_top.u_cpu.wdata = 16'h5555;
39
40            repeat (4) begin
41                #10
42                {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} =
43                    {u_top.u_cpu.write_enable, u_top.u_cpu.read_enable} + 2'b01;
44            end
45        end
46        #50 $finish;
47    end
48
49 endmodule
```

- TB output:

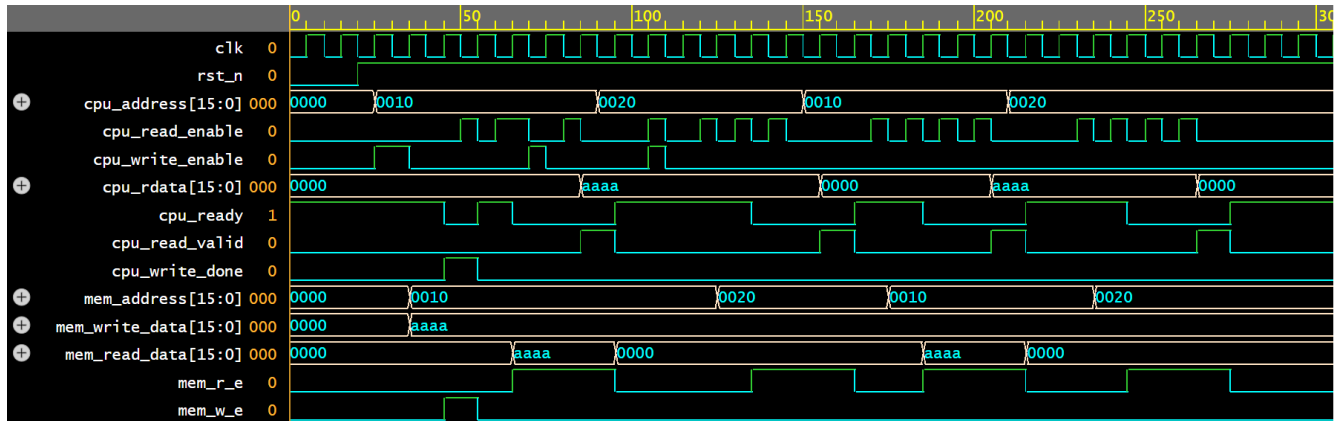
```

Time=20 Addr=0000 WData=0000 RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=25 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=45 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=1
Time=55 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=65 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=0
Time=85 Addr=0010 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=90 Addr=0020 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=95 Addr=0020 WData=aaaa RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=100 Addr=0020 WData=5555 RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=135 Addr=0020 WData=5555 RData=aaaa Ready=0 ReadValid=0 WriteDone=0
Time=150 Addr=0010 WData=5555 RData=aaaa Ready=0 ReadValid=0 WriteDone=0
Time=155 Addr=0010 WData=5555 RData=0000 Ready=0 ReadValid=1 WriteDone=0
Time=160 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=1 WriteDone=0
Time=165 Addr=0010 WData=aaaa RData=0000 Ready=1 ReadValid=0 WriteDone=0
Time=185 Addr=0010 WData=aaaa RData=0000 Ready=0 ReadValid=0 WriteDone=0
Time=205 Addr=0010 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=210 Addr=0020 WData=aaaa RData=aaaa Ready=0 ReadValid=1 WriteDone=0
Time=215 Addr=0020 WData=aaaa RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=220 Addr=0020 WData=5555 RData=aaaa Ready=1 ReadValid=0 WriteDone=0
Time=245 Addr=0020 WData=5555 RData=aaaa Ready=0 ReadValid=0 WriteDone=0
Time=265 Addr=0020 WData=5555 RData=0000 Ready=0 ReadValid=1 WriteDone=0
Time=275 Addr=0020 WData=5555 RData=0000 Ready=1 ReadValid=0 WriteDone=0
$finish called from file "testbench.sv", line 46.
$finish at simulation time 310

```

Complete System Simulation Screenshots

- All controller signals:



Conclusion

In this project, we designed and verified a simplified memory controller that manages read and write operations between a CPU and memory. The controller was implemented using a basic FSM with idle, read, and write states, and it correctly prioritizes write operations when both read and write are requested. A simple CPU model was created to generate requests, and a memory module was added to complete the system. Testbenches were developed for both the controller and the full system to automatically check correctness through simulation. The results showed that the design behaves as expected, with proper handshaking using `ready`, `read_valid`, and `write_done` signals. Overall, the project met its main goals and provided hands-on practice with RTL design, simulation, and verification of digital systems.